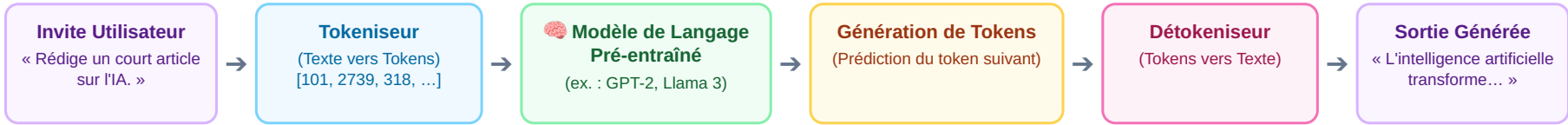


Projet d'IA Générative

Génération de Texte à l'aide d'un LLM Pré-entraîné



Objectif du Projet :

Générer du texte cohérent et naturel, comparable au langage humain, grâce à un grand modèle de langage (LLM) pré-entraîné.

Fonctionnement :

- Invite d'entrée (Prompt) :** Vous fournissez une invite textuelle.
- Tokenisation :** Le texte est converti en tokens (représentation numérique).
- Modèle pré-entraîné :** Un grand modèle de langage prédit le token suivant en fonction du contexte.
- Décodage :** Les tokens prédits sont reconvertis en texte lisible.

Options de Modèles :

- Modèles OpenAI (via API)
- Meta Llama (via Hugging Face)
- Mistral, Gemma, Phi, etc.

Concepts Clés :

- Architecture Transformer (auto-attention)
- Pré-entraîné sur de très grands volumes de données textuelles
- Génère le texte séquentiellement, token par token
- Échantillonnage probabiliste (ex. : température, top-k)

Métriques d'Évaluation (génération de texte) :

- Perplexité** (plus elle est basse, meilleur est le modèle)
- BLEU / ROUGE** (lorsqu'un texte de référence existe)
- Évaluation humaine** (fluidité, pertinence, cohérence)

Applications :

- Chatbots & assistants virtuels
- Création de contenu (articles, e-mails, scripts)
- Génération de code informatique
- Synthèse et résumé automatique
- Écriture créative

L'IA générative peut créer de nouveaux contenus (texte, code, images, audio) en apprenant des structures à partir de grandes quantités de données.

Projet d'IA Générative : Code Python

```
# Installer les bibliothèques requises (décommenter si besoin)
# !pip install transformers torch

from transformers import AutoTokenizer, AutoModelForCausalLM

def charger_modele(nom_modele="gpt2"):
    """Charge le tokeniseur et le modèle."""
    tokeniseur = AutoTokenizer.from_pretrained(nom_modele)
    modele = AutoModelForCausalLM.from_pretrained(nom_modele)
    return tokeniseur, modele

def generer_texte(invite, modele, tokeniseur, max_nouveaux_tokens=100,
                  temperature=0.8, top_k=50):
    """Génère du texte à l'aide du modèle."""
    # Tokeniser l'entrée
    entrees = tokeniseur(invite, return_tensors="pt")

    # Générer la sortie
    sorties = modele.generate(
        **entrees,
        max_new_tokens=max_nouveaux_tokens,
        temperature=temperature,
        top_k=top_k,
        do_sample=True,
        pad_token_id=tokeniseur.eos_token_id
    )

    # Décoder les tokens en texte
    texte_genere = tokeniseur.decode(sorties[0], skip_special_tokens=True)
    return texte_genere

# Programme principal
if __name__ == "__main__":
    nom_modele = "gpt2" # Tester d'autres modèles : "meta-llama/Meta-Llama-3-8B"
    tokeniseur, modele = charger_modele(nom_modele)
    invite = "Rédige un court blog sur les atouts de l'apprentissage de l'IA."
    resultat = generer_texte(invite, modele, tokeniseur)

    print("\nTexte Généré :\n")
    print(resultat)
```

```
# Exemple

invite = "Rédige un court blog sur les atouts de l'apprentissage de l'IA."
sortie = generer_texte(invite, modele, tokeniseur)
print(sortie)
```

Exemple de résultat (les sorties peuvent varier)

Texte Généré :

L'apprentissage de l'IA ouvre un champ infini d'opportunités. Il permet de résoudre des problèmes concrets, d'automatiser des tâches répétitives et de bâtir des solutions innovantes. Ces compétences ouvrent des carrières captivantes en science des données et ingénierie logicielle, tout en vous permettant de rester à la pointe des évolutions technologiques.

Vous pouvez également essayer :

- Changer de modèle : "meta-llama/Meta-Llama-3-8B", "mistralai/Mistral-7B-Instruct", "google/gemma-7b"
- Ajuster les paramètres : temperature, top_k, max_new_tokens
- Utiliser une invite conversationnelle (ex. : chatbot)
- Créer une application web simple avec Streamlit

En quelques lignes de code seulement, vous pouvez exploiter un LLM pré-entraîné pour générer du texte de qualité humaine adapté à de multiples applications pratiques.